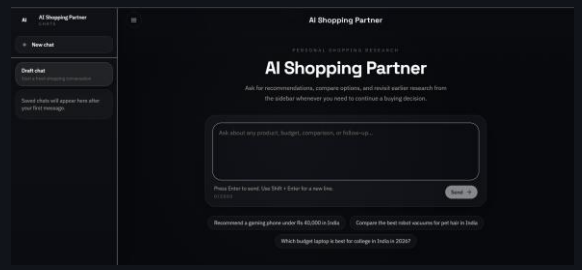
PROJECT REVIEW DECK

Agentic AI Personal Shopper

- Full-stack AI shopping assistant for evidence-grounded product decisions
- Built with React, Flask, Gemini, YouTube transcript research, SSE, and Postgres
- Review scope: vision, architecture, engineering journey, deployment, and outcomes

Item	Details
Names	V Rama Chandra Chathurvedi, Aravind Prasad R
Reg. No.	25MCS0067, 25MCS0060
Guide	Dr. Baiju B V
Project	AI Shopping Partner



Final landing experience of the deployed web application

Executive Summary and Core Ideologies

Core ideology	How it was implemented	Review value
Evidence-grounded shopping	Recommendations synthesized from YouTube review transcripts and explicit source links	Moves output away from generic chatbot advice
Conversational continuity	Follow-up questions, saved sessions, auto-titles, rename/delete, sidebar revisit flow	Lets users continue buying decisions over time
Latency-conscious engineering	ThreadPoolExecutor, SSE streaming, hybrid tool routing, lighter transcript payloads, caching	Improves both real and perceived responsiveness
Privacy-first persistence	Anonymous browser-scoped ownership for each session	Prevents chat leakage on a public website
Deployment realism	GitHub source control, Render Blueprint, Postgres, env wiring, health checks	Makes the project review-ready beyond localhost

- Overall design target: a practical AI shopping product, not a toy demo.

Problem Statement and Project Goals

Problem being solved

- Online product research is fragmented across videos, specs, comments, and opinions.
- Buyers need verdicts and comparisons, not just a list of links.
- Typical chatbots often lack cited evidence and strong session continuity.
- Public deployment introduces privacy and persistence challenges.

Project goals

- Accept natural-language shopping requests and follow-up questions.
- Search YouTube reviews only when fresh evidence is required.
- Extract transcripts automatically and synthesize a shopping verdict.
- Stream responses live and save research for later reuse.
- Deploy publicly with a professional UI and production-style persistence.

Development Timeline: From MVP to Public Deployment

Phase	What was built	Outcome
1. Foundation	React/Vite frontend shell + modular Flask backend (app.py, agent.py, scraper.py)	Stable starting architecture
2. Retrieval + AI	YouTube search, transcript extraction, Gemini synthesis, SSE streaming	End-to-end chat research flow
3. Product UX	Dark interface, prompt composer, source chips, better output formatting	Professional and review-friendly UI
4. Session features	Saved chats, smart title generation, rename/delete, sidebar navigation	Conversation continuity
5. Hardening	Privacy isolation, latency fixes, model fallback, Render/Postgres deployment	Publicly usable system

- The project evolved from a functional prototype into a production-style full-stack application.
- Each iteration addressed a real engineering limitation: reliability, speed, privacy, or UX clarity.

Initial Scope vs Final Delivered System

Area	Initial requirement	Final implementation	Status
Backend structure	app.py, agent.py, scraper.py	Added storage.py, session CRUD routes, privacy ownership, deployment-ready configuration	Delivered
Intent routing	Fast decision on whether fresh YouTube research is needed	Hybrid fast-path research + Gemini tool decision for ambiguous turns	Delivered
Parallel retrieval	Fetch top video transcripts in parallel	Multi-variant search + dedupe + ThreadPoolExecutor transcript extraction	Delivered
Streaming	Gemini response streamed to UI	SSE with session, status, sources, chunk, done, and error events	Delivered
Chat UI	Modern conversational interface	Dark minimal interface with collapsible sidebar and keyboard-first composer	Delivered

Technology Stack and Platform Choices

Layer	Technology	Purpose	Why this choice
Frontend	React + Vite + Tailwind CSS + react-markdown	Interactive chat UI and fast development	Component-based, responsive, lightweight, markdown-friendly
Backend API	Python Flask + Gunicorn	REST API, SSE streaming, routing	Simple, modular, and suitable for Render web services
AI layer	Google Gemini Flash family	Product verdicts, comparisons, titles	Fast enough for conversational synthesis
Retrieval	youtube-search-python + youtube-transcript-api	Review discovery and evidence extraction	No official YouTube Data API keys required
Persistence	SQLite locally, Postgres on Render	Session and message storage	Easy local setup + durable production deployment
Deployment	GitHub + Render Blueprint	Public hosting and infra wiring	Straightforward for Flask, SSE, and Postgres

System Architecture and End-to-End Flow

Stage	Responsible layer	Output / behavior
1. Prompt entry	React frontend	Captures query, updates local state, opens/uses active session
2. Chat request	Flask POST /api/chat	Creates or resumes a session and starts streaming events
3. Routing	agent.py	Decides follow-up vs fresh YouTube research
4. Evidence retrieval	scraper.py	Search variants + parallel transcript extraction + source list
5. Synthesis	Gemini	Generates verdict, compare blocks, watch-outs, and INR pricing cues
6. Streaming	SSE	Delivers status, source chips, and text chunks in real time
7. Persistence	storage.py	Stores messages, sessions, titles, and owner-scoped visibility
8. Revisit	Sidebar + session API	Lets user reopen, rename, or delete research threads

- SSE event contract used by the UI: session, status, sources, chunk, done, and error.
- Architecture intentionally separates routing, retrieval, synthesis, storage, and presentation concerns.

Backend Module Design

Module	Primary responsibilities	Notable implementation details
app.py	Flask setup, CORS, routes, SSE streaming, session endpoints	Owns POST /api/chat, health checks, title scheduling, and cookie identity
agent.py	Gemini orchestration and shopping reasoning	Hybrid routing, tool wrapper, model fallback, response policy, title generation
scraper.py	YouTube search and transcript extraction	India-focused query shaping, transcript fallback logic, dedupe, caching support
storage.py	DB schema and CRUD	SQLite local fallback, Postgres production support, owner-scoped session filtering

- The modular backend kept routing, research, model logic, and persistence independently maintainable.
- This structure also made it easier to debug individual issues such as quota failures, transcript failures, and chat leakage.

YouTube Retrieval Engine and Gemini Intelligence

YouTube evidence pipeline

- Derived search variants from the user query with India- and review-oriented bias.
- Searched multiple candidate videos and removed duplicates.
- Fetched transcripts in parallel with ThreadPoolExecutor for lower latency.
- Skipped videos that raised TranscriptsDisabled or NoTranscriptFound.
- Used transcript fallbacks, trimming, and caching before passing evidence to Gemini.

Gemini orchestration strategy

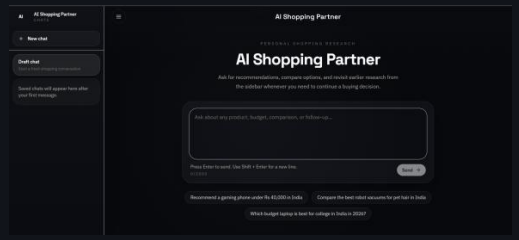
- Wrapped YouTube research as a native tool: fetch_youtube_reviews(product_name).
- Used a hybrid router: fast-path for obvious new research, model decision for ambiguous or follow-up turns.
- Streamed final responses through SSE for a live typing effect.
- Added stronger prompts for specific models, INR budgets, comparisons, and watch-outs.
- Introduced model fallback and quota-aware behavior for public deployment.

Frontend UX and Interaction Design

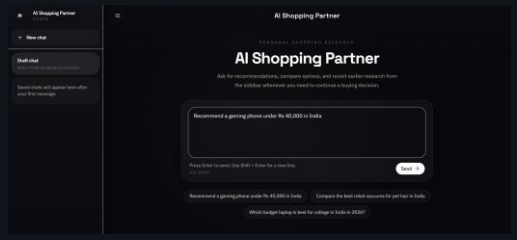
UX objective	Final implementation
Minimal visual noise	Dark, low-glare interface with centered landing page and restrained accent colors
Fast first interaction	Suggestion chips, large composer, keyboard-first prompt flow
Streaming clarity	Live status labels, source chips, incremental answer rendering
Navigation	Collapsible sidebar, home click behavior, draft/new chat flow
Session control	Rename, delete, revisit, and single smart auto-title generation
Responsiveness	Desktop sidebar plus mobile-friendly drawer behavior

- The UI deliberately moved away from cluttered tables and duplicated controls.
- The final design aligns more closely with the interaction patterns of ChatGPT, Gemini, Claude, and Copilot.

UI Walkthrough: Landing State and Prompt Entry



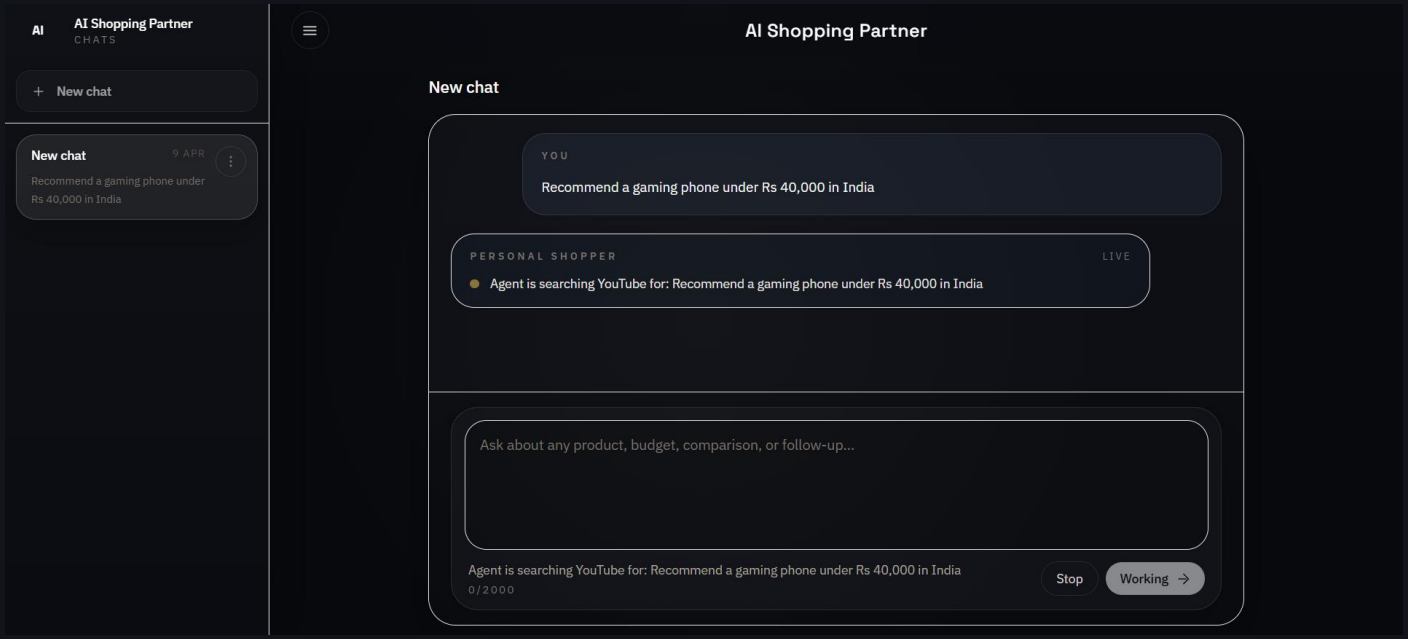
Home view before a prompt is sent



Prompt drafted for a specific Indian-market shopping query

- Clean landing page with one clear call to action: ask a shopping question.
- Prompt suggestions accelerate first use and demonstrate supported query styles.
- Composer supports Enter-to-send and Shift+Enter for multiline prompts.

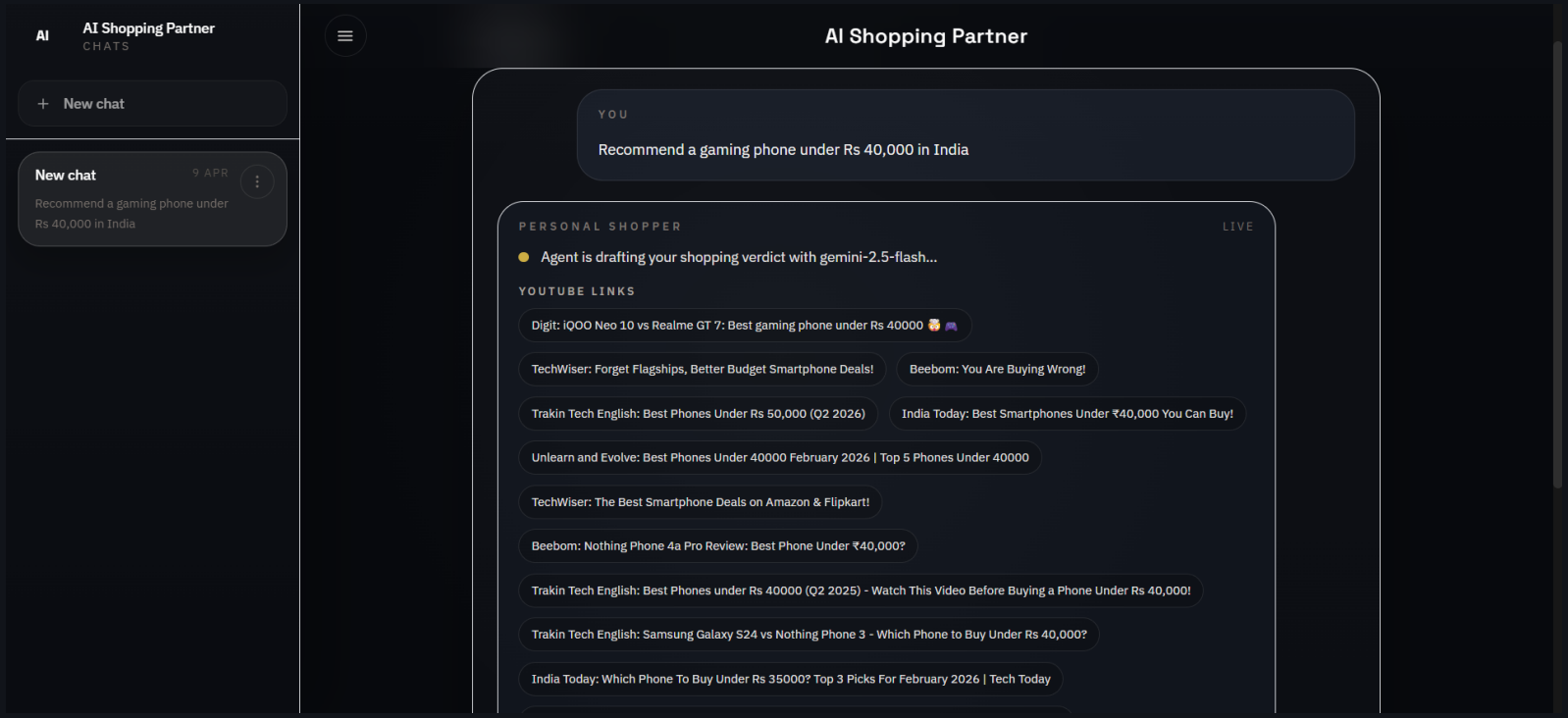
UI Walkthrough: Live Research and Streaming States



Thread begins with the user message and immediately surfaces live research status

State	Meaning
Searching YouTube	Retrieval is running and transcript evidence is being collected
Live	Assistant stream is active and the response channel is open

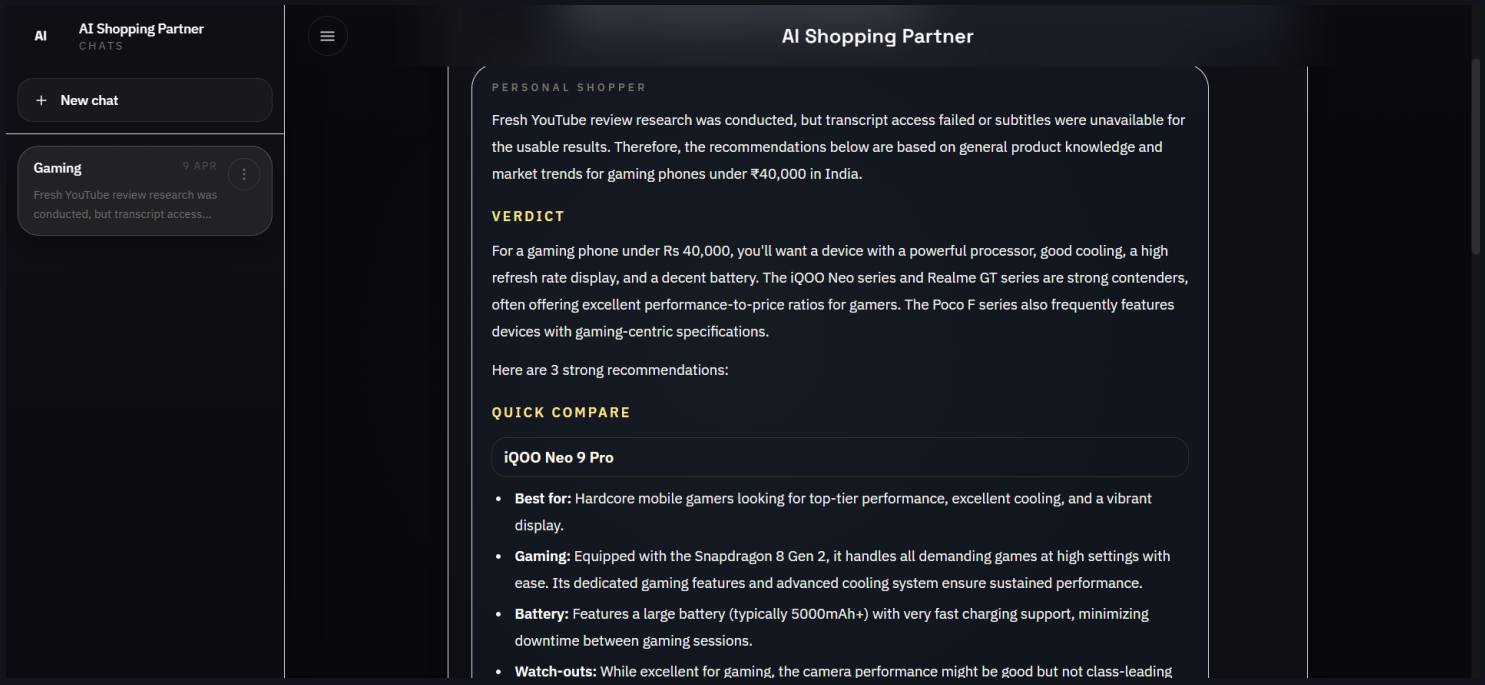
UI Walkthrough: Source Transparency During Generation



Source links are surfaced while the verdict is still being drafted

- Multiple review channels contribute evidence instead of relying on a single video.
- Users can see exactly which review videos informed the answer before the response completes.
- This improves trust, explainability, and review auditability.

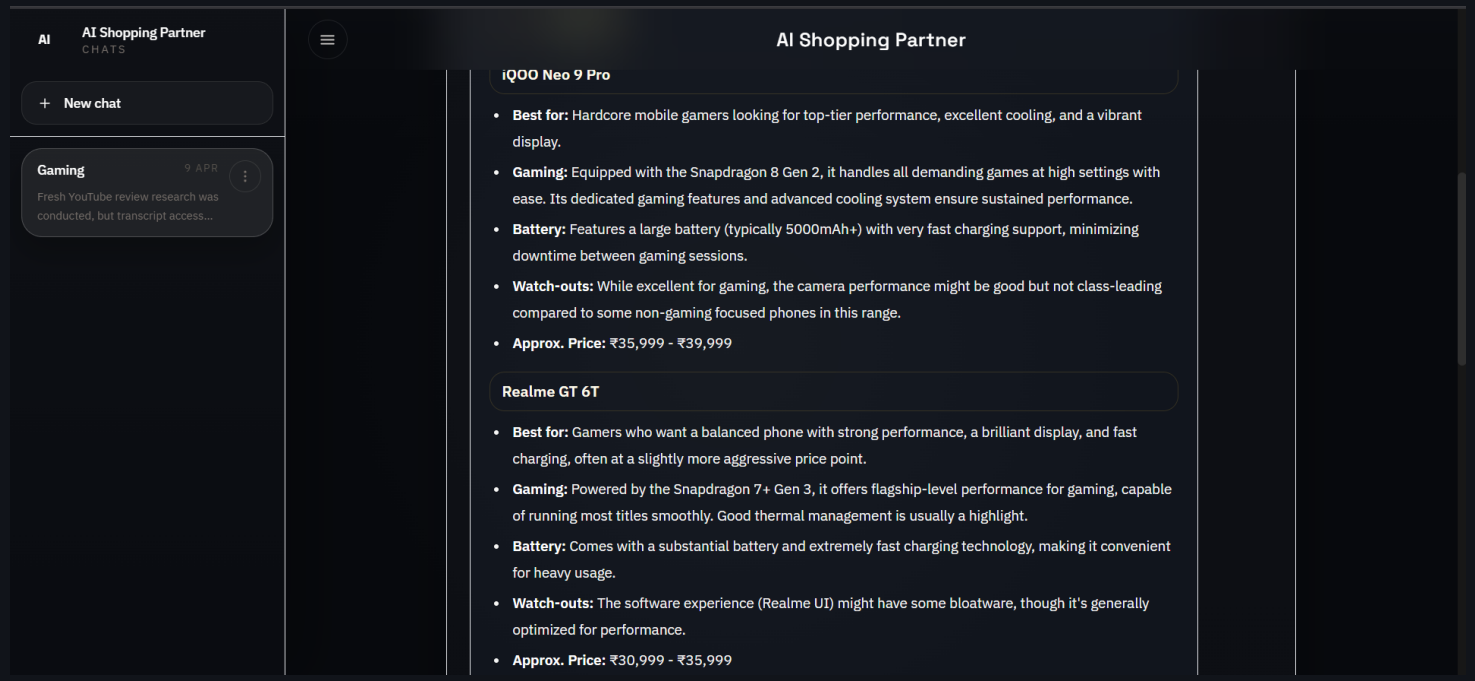
Recommendation Output: Verdict and Quick Compare



Final answer format emphasizes verdict, quick compare blocks, watch-outs, and pricing

- The output moved away from clumsy markdown tables toward compact compare blocks.
- Each recommendation includes best-for guidance, gaming/performance notes, battery, watch-outs, and approximate INR pricing.
- When transcript access is limited, the assistant degrades gracefully instead of failing outright.

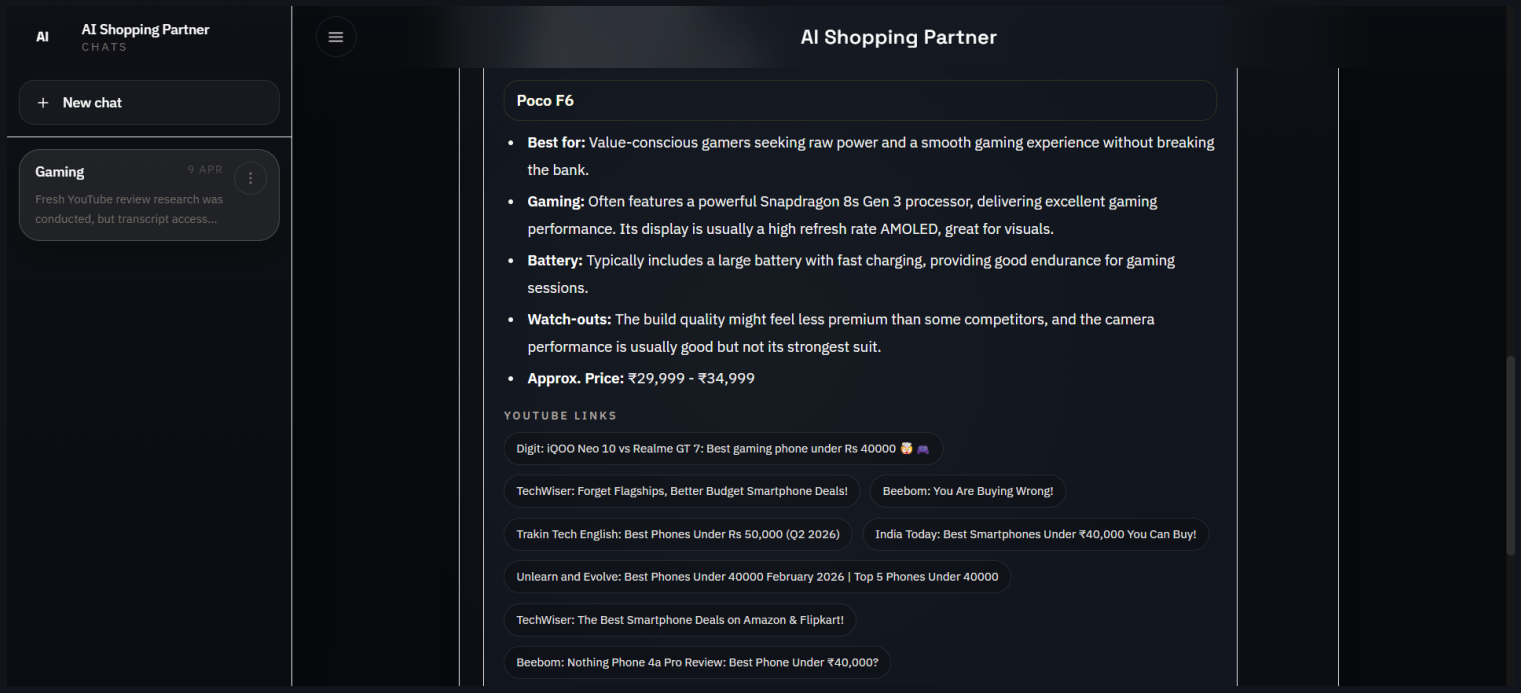
Recommendation Output: Product-Level Specificity



Recommendations are presented as concrete product options rather than brand-only advice

- The response policy was strengthened to prefer exact models and budget-relevant shortlists.
- Ambiguous categories can trigger clarifications instead of generic recommendations.
- India-focused pricing improves practical decision support for the target audience.

Recommendation Output: Attached Evidence Links



Source chips remain attached to the recommendation output for quick verification

- Final answers keep source visibility close to the recommendation itself.
- This supports follow-up questions such as battery, thermals, value, or camera trade-offs.
- The UI is designed to feel conversational while still preserving evidence grounding.

Sessions, Privacy, and Persistence

Capability	Final behavior	Engineering mechanism
Saved sessions	Reopen earlier chats from the sidebar	Session and message tables in DB
Rename / delete	Users control chat titles and cleanup	PATCH/DELETE session endpoints + sidebar actions
Single smart auto-title	One refined title appears after the first answer	Background title generation after assistant response
Browser-private chats	One visitor cannot see another visitor's conversations	Anonymous cookie + owner_id session filtering
Local and production storage	SQLite locally, Postgres on Render	DB_PATH fallback locally, DATABASE_URL in production

- A critical late-stage fix was preventing public chat leakage once the site was deployed on a shared URL.
- Authentication was intentionally avoided to keep the product lightweight, so privacy was implemented per browser session instead.

Deployment Architecture and Production Setup

Deployment layer	What was done	Outcome
Source control	Initialized Git repository, created GitHub repo, documented setup in README	Reproducible project history
Backend hosting	Deployed Flask API to Render Web Service with Gunicorn	Public SSE-capable API
Frontend hosting	Deployed Vite build to Render Static Site	Public client URL
Database	Provisioned Render Postgres and injected DATABASE_URL	Durable session persistence in production
Configuration	Wired GEMINI_API_KEY, CORS_ORIGIN, VITE_API_BASE_URL, health checks	Connected services and prevented CORS issues
Operational fit	Chose Render over Vercel for Flask + SSE + long-lived backend patterns	Simpler deployment model for this architecture

- Production persistence was upgraded from SQLite to Postgres because free service filesystems are not durable enough for shared deployment.
- A Render Blueprint (render.yaml) keeps frontend, backend, and database wiring aligned.

Major Technical Issues and Their Resolutions

Issue	Observed problem	Resolution
PowerShell execution policy	Blocked venv activation and direct npm usage	Used .venv\\Scripts\\python and npm.cmd instead of changing global policy
Python dependency mismatch	youtube-transcript-api 0.6.3 failed on Python 3.14	Upgraded to a compatible version
YouTube search breakage	youtube-search-python conflicted with newer httpx	Pinned a compatible httpx release
Gemini model / config errors	Unavailable model aliases and invalid function calling config	Updated model defaults, added fallback logic, corrected tool config
Quota exhaustion	Public traffic exceeded free-tier request limits	Added model fallback, better usage patterns, and quota-aware messaging

- These issues were part of turning the app from a local prototype into a public deployment.

Major Product and UX Issues and Their Resolutions

Issue	Observed problem	Resolution
Public chat leakage	Shared URL exposed saved chats to other users	Introduced browser-scoped anonymous ownership filtering
Generic recommendations	Queries like watches under 5000 produced brand-level answers	Strengthened prompts, added category clarification, pushed model-specific output
Slow first responses	Fresh research paid for multiple AI/retrieval costs	Hybrid fast-path routing, leaner transcripts, candidate tuning, short-lived cache
Vague chat titles	Auto-titles were not useful in the sidebar	Moved to one smarter title generated after the first answer
Cluttered UI	Duplicate controls and poor comparison formatting reduced clarity	Redesigned the interface, simplified controls, replaced tables with compare blocks

- The final user experience is the result of iterative correction, not just initial implementation.

Final Feature Set and Review Readiness

Capability group	Delivered features
Core AI	Conversational shopping assistant, follow-up support, Gemini synthesis, India-focused outputs
Retrieval	YouTube review search, transcript-backed reasoning, parallel transcript extraction, source dedupe, caching
Streaming UX	SSE response streaming, live status updates, source chips, incremental rendering
Sessions	Saved chats, smart titles, rename/delete, sidebar revisit flow
Privacy	Browser-scoped session isolation with no cross-user leakage
Deployment	GitHub repo, Render internal backend, Postgres persistence, health checks
UI polish	Dark minimal design, collapsible sidebar, centered landing page, keyboard-first interaction

- Final result: a publicly deployed, evidence-grounded AI shopping assistant with real engineering depth across retrieval, AI orchestration, UX, privacy, persistence, and deployment.
- The system is ready for project review because it demonstrates both product thinking and end-to-end implementation rigor.